Преобразования яркости изображений и пространственная фильтрация

1. Функции преобразования яркости изображений

Когда приходится работать с полутоновыми изображениями, иногда необходимо изменять значения яркости изображений. Например, необходимо поменять значения яркости черного и белого цвета или сделать темнее темные участки и светлее светлые. Преобразования яркости изображения применяются с целью выделения интересуемых областей на изображении, с помощью увеличения контрастности между определенными значениями яркости. Например, следующие два изображения отображают информацию до и после преобразования яркости. Первоначально, куртка оператора казалась черной, но с преобразованием яркости, разница между черными значениями яркости, которые были слишком близки прежде, была увеличена так, чтобы пуговицы и карманы стали видимыми.

Исходное	**После преобразования яркости**
	

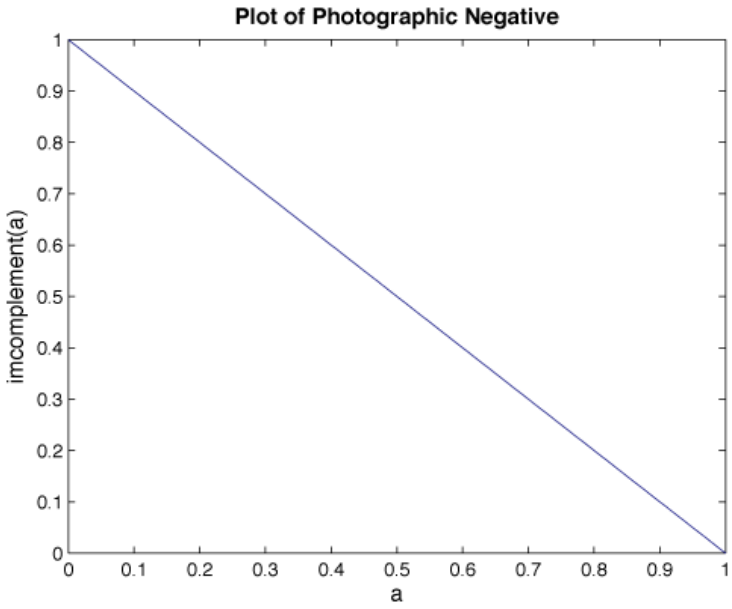
Как правило, внесение изменений в яркость осуществляется с помощью функций преобразования яркости (Intensity Transformation Functions). В следующих пунктах будем говорить о четырех основных функциях преобразования яркости:

1. фотографический негатив (используя `imcomplement`);
2. гамма-преобразования (используя `imadjust`);
3. логарифмические преобразования (используя `c*log(1+f)`);
4. преобразования растяжения контрастности (используя `1./(1+(m./(double(f)+eps)).^E`).

1.1. Фотографический негатив

Этот тип обработки осуществляется с использованием негативного преобразования и особенно подходит для усиления белых или серых деталей на фоне темных областей изображения, особенно когда темные области имеют преобладающие размеры.

Фотографический негатив, вероятно, самое простое для описания преобразование яркости. Предположим, что мы работаем с двойными (double) массивами оттенков серого, где черный это 0 и белый это 1. Идея состоит в том, чтобы 0 стал 1, а 1 стал 0, и любые градиенты между ними также инвертированы. В терминах яркости это означает, что истинно черный цвет становится истинно белым и наоборот. MATLAB имеет функции для создания фотографических негативов – `imcomplement(f)`. Учитывая `a = 0:.01:1`, ниже приведен график сопоставление исходных значений (ось абсцисс) и функции `imcomplement`.



Ниже приведен пример фотографического негатива. Обратите внимание, что теперь можно прочитать надпись в середине колеса лучше, чем было раньше:

Исходное	Фотографический негатив
	

Код системы MATLAB, который позволяет создать эти два изображения следующий:

```
I = imread('tire.tif');
imshow(I);
J = imcomplement(I);
figure, imshow(J);
```

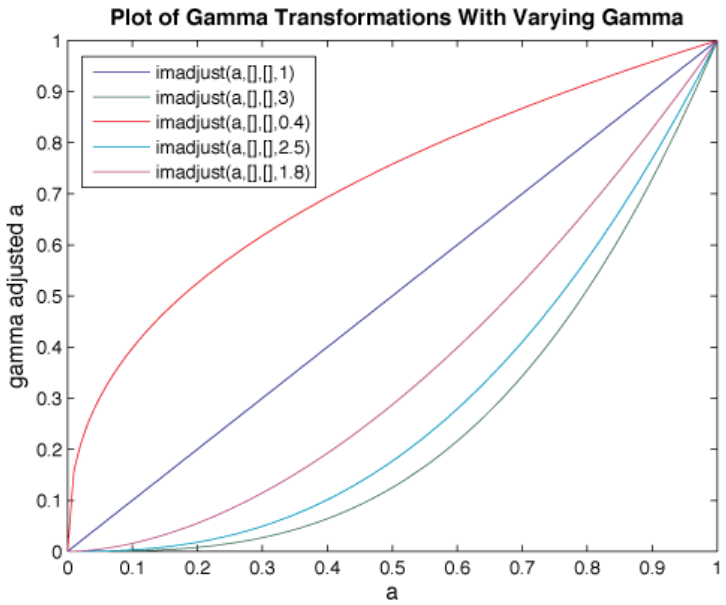
1.2. Гамма-преобразования

С гамма-преобразованиями, можно задавать форму яркостной кривой яркости для полутоновых изображений, отображающей яркость `f` в яркость `g`: яркость отображения смещается вверх в сторону более ярких значений (когда `gamma` меньше 1) или вниз в сторону менее ярких значений (при `gamma` больше 1).Функция MATLAB, которая создает эти гамма-преобразования:

```
g = imadjust(f, [low_in high_in], [low_out high_out], gamma)
```

`f` является исходным изображением, `gamma` контролирует форму кривой, и `[low_in high_in]` и `[low_out high_out]` используется для отсечения (ограничения). Значения ниже `low_in` обрезаются до `low_out` и значения выше `high_in` обрезаются до `high_out`. Для достижения целей этой лабораторной работы будем использовать `[]` для обоих `[low_in high_in]` и `[low_out high_out]`. Это означает, что полный диапазон

входных значений отображается на весь выходной спектр. Учитывая, что `a = 0:.01:1`, следующий график показывает влияние гамма-преобразования с различными значениями параметра `gamma`. Обратите внимание, что красная линия имеет значение параметра `gamma = 0.4`, который создает верхнюю кривую и изображение становится ярче.



Ниже приведены результаты трех гамма-преобразований, показанных на графике выше. Обратите внимание, что значения большие 1 создают темное изображение, в то время как значения между 0 и 1, создают яркое изображение с большим контрастом в темных местах, так что вы можете увидеть детали на автомобильных колесах.

Исходное (`gamma = 1`)	`gamma = 3`	`gamma = 0.4`
		

Код системы MATLAB, который позволяет создать эти три изображения следующий:

```
I  = imread('tire.tif');
J  = imadjust(I,[],[],1);
J2 = imadjust(I,[],[],3);
J3 = imadjust(I,[],[],0.4);
imshow(J);
figure, imshow(J2);
figure, imshow(J3);
```

Гамма-преобразования являются важной частью процесса отображения изображений.

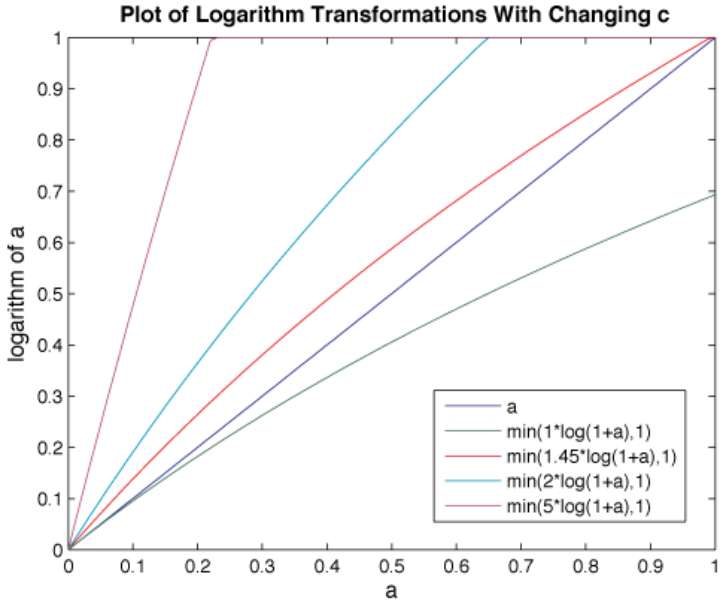
1.3. Логарифмические преобразования

Логарифмические преобразования являются основными инструментами манипуляций в динамическом диапазоне и могут быть использованы для повышения яркости изображений (подобно, гамма-преобразованию, где `gamma < 1`). Чаще всего они используются для повышения детализации (или контрастности) на более низких значениях яркости. Они

особенно полезны для выявления деталей в преобразованиях Фурье (рассматриваются в следующих лабораторных работах). Основное применение логарифмического преобразования состоит в сжатии динамического диапазона. В MATLAB, уравнение, используемое для получения логарифмического преобразования изображения `f`, является:

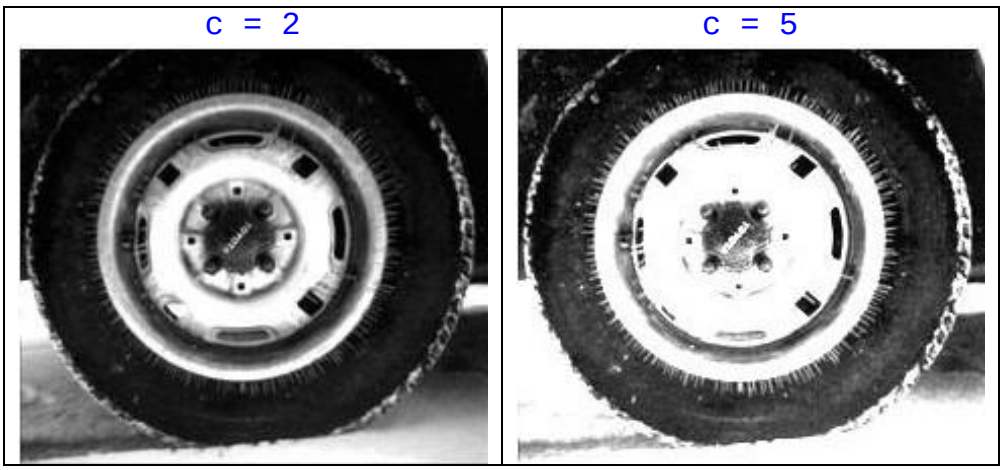
```
g = c * log(1 + double(f))
```

Постоянная `c`, как правило, используется для масштабирования диапазона функции `log` в соответствии с входной областью. В этом случае `c = 255/log(1 + 255)` для `uint8` изображения, либо `c = 1/log(1 + 1)(~1.45)` для `double` изображения. Она также может быть использована для дальнейшего увеличения контрастности – чем выше `c`, тем ярче изображение. Форма этого преобразования похожа на гамма-кривую, где нижние значения отображаются в 0, а верхние – в 1. Отметим, однако, что форма гамма-кривой является переменной, в то время как форма функции `log` зафиксирована. Учитывая, что `a = 0:.01:1`, график ниже отображает результаты для различных значений `c`. У-значения ограничены значением 1 функцией `min` для графика при `c = 2` и `c = 5` (зеленовато-голубая и пурпурная линии соответственно).



Ниже показано исходное изображение, а также результаты применения трех приведенных выше преобразований. Обратите внимание, что при `c = 5`, изображение самое яркое и можно увидеть радиальные линии на внутренней стороне колеса (эти линии едва видимы в исходном изображении, потому что не хватает контрастности в нижних значениях яркости).

Исходное	`c = 1`
	

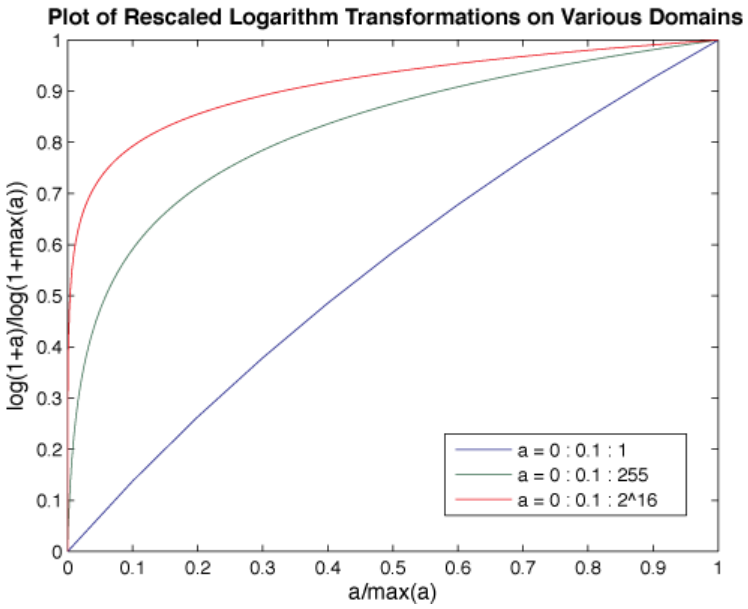


Код системы MATLAB, который позволяет создать эти изображения следующий:

```
I  = imread('tire.tif');
imshow(I);
I2 = im2double(I);
J  = 1 * log(1 + I2);
J2 = 2* log(1 + I2);
J3 = 5* log(1 + I2);
figure, imshow(J);
figure, imshow(J2);
figure, imshow(J3);
```

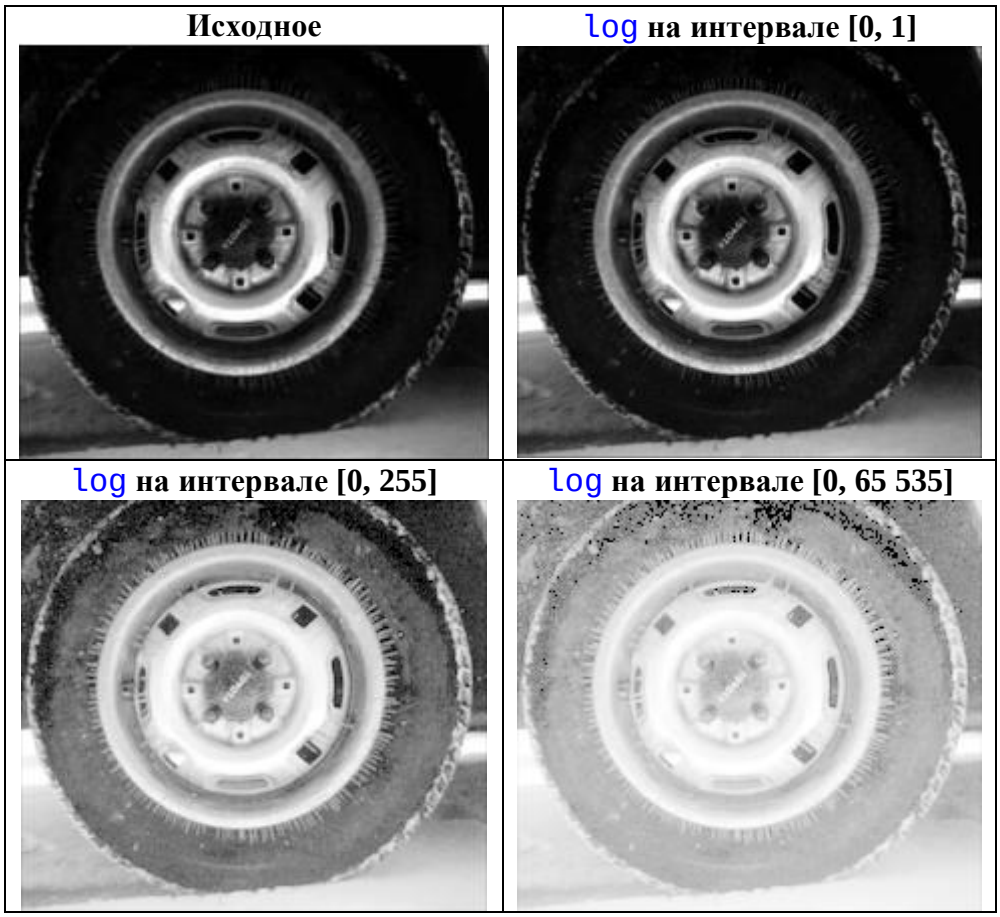
Обратите внимание на потери деталей в светлых областях, где значения интенсивности зафиксированы (ограничены). Любые значения больше единицы, производимые масштабированием, отображаются как равные значению 1 (полная яркость) и должны быть ограничены. Фиксация (ограничение) в MATLAB может быть выполнена с помощью `min(matrix, upper_bound)` и `max(matrix, lower_bound)` функциями, как показано в легенде на графике выше.

Хотя логарифмы могут быть вычислены в различных основаниях с помощью встроенных в MATLAB `log10`, `log2` и `log` (натуральный логарифм), результирующая кривая, когда диапазон масштабируется в соответствии с областью, является одинаковой для всех оснований. Ниже приведены примеры `log` кривой для нескольких диапазонов входных значений:



Важно знать это, если вы планируете эффективно использовать логарифмическое

преобразование. Ниже представлен результат масштабирования значений изображения для этих диапазонов перед применением логарифмического преобразования:



Код системы MATLAB, который позволяет создать эти изображения следующий:

```
tire = imread('tire.tif');
d = im2double(tire);
figure, imshow(d);
%log on domain [0, 1]
f = d;
c = 1/log(1+1);
j1 = c*log(1+f);
figure, imshow(j1);
%log on domain [0, 255]
f = d*255;
c = 1/log(1+255);
j2 = c*log(1+f);
figure, imshow(j2);
%log on domain [0, 2^16]
f = d*2^16;
c = 1/log(1+2^16);
j3 = c*log(1+f);
figure, imshow(j3);
```

Отметим, что для интервала [0, 1], эффект от логарифмического преобразования едва заметен, а для интервала [0, 65535] эффект чрезвычайно преувеличен. Также отметим, что, в отличие от линейного масштабирования и ограничения, крупные детали по-прежнему отображаются в светлых областях.

1.4. Преобразования растяжения контрастности

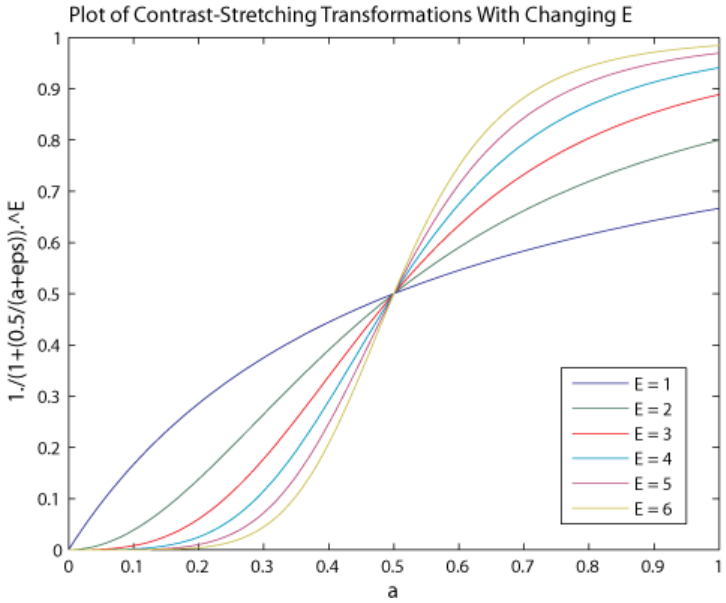
Преобразования растяжения контрастности являются основными инструментами

манипуляций в динамическом диапазоне. Они увеличивают контрастность между темным и светлым. Во второй лабораторной работе мы видели упрощенный вариант автоматической регулировки контрастности. В результате такого преобразования пиксели изображения с минимальным значением яркости отображаются как черные, а с максимальным значением – как белые. Все остальные принимают промежуточное значение яркости. Если преобразование линейное, то значения яркости будут пропорциональными. Возможно также и нелинейные преобразования, которые обладают большими возможностями по улучшению изображений. Таким образом, следует говорить о растяжении динамического диапазона значений яркостей пикселей изображения, с помощью которого можно регулировать контрастность изображения.

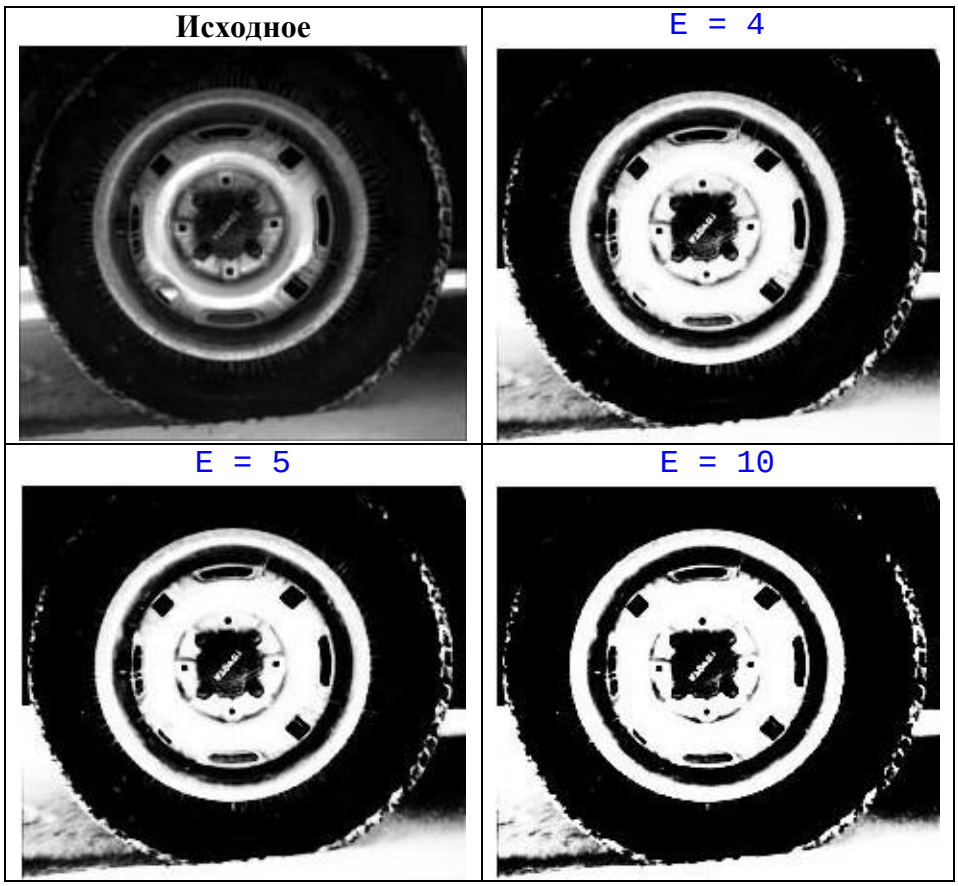
Иногда необходимо растянуть интенсивность вокруг определенного уровня. В результате таких преобразований минимальные значения яркостей исходного изображения принимают минимально возможное значение, а максимальное значения яркостей исходного изображения принимают, соответственно, максимально возможное значение в рамках данного диапазона. Для создания такого преобразования растяжения контрастности в MATLAB можно использовать следующие функции:

```
g = 1./(1 + (m./(double(f) + eps)).^E)
```

`E` контролирует наклон функции и `m` – средняя линия, от который будет происходить переход от темных оттенков к светлым. `eps` – константа MATLAB, которая является расстоянием между 1.0 и следующим самым большим числом, которое может быть представлено в виде вещественного числа с плавающей точкой двойной точности. Заметим, что использование данной величины позволяет избежать ошибки переполнения, если в `f` имеются нулевые величины. Ниже представлено два графика/диаграммы, отображающих результаты изменения `m` и `E`. Приведенный ниже график показывает результаты для нескольких различных значений `E`, при `a = 0:.01:1` и `m = 0.5`.



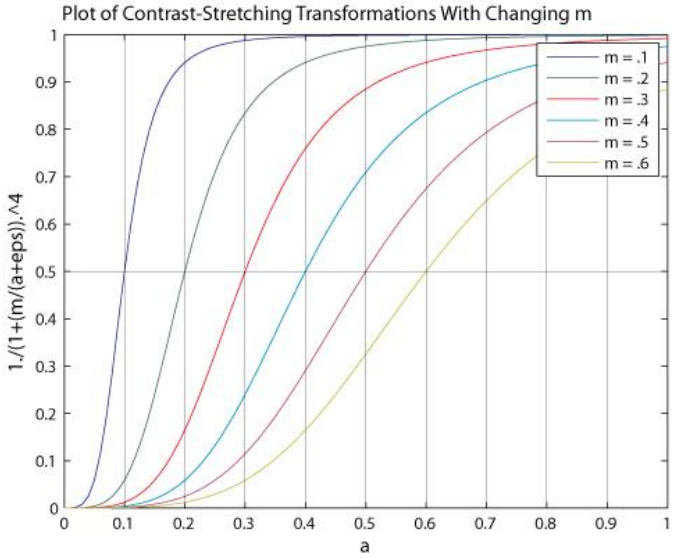
Ниже показано исходное изображение, а также результаты применения трех преобразований, приведенных выше. Значение `m`, используемое ниже, представляет собой среднее значение яркости изображения (0.2104). При очень высоких значений `E`, таких как 10, функция становится больше похожа на пороговую функцию с порогом `m` – полученное изображение больше черно-белое, чем серое.



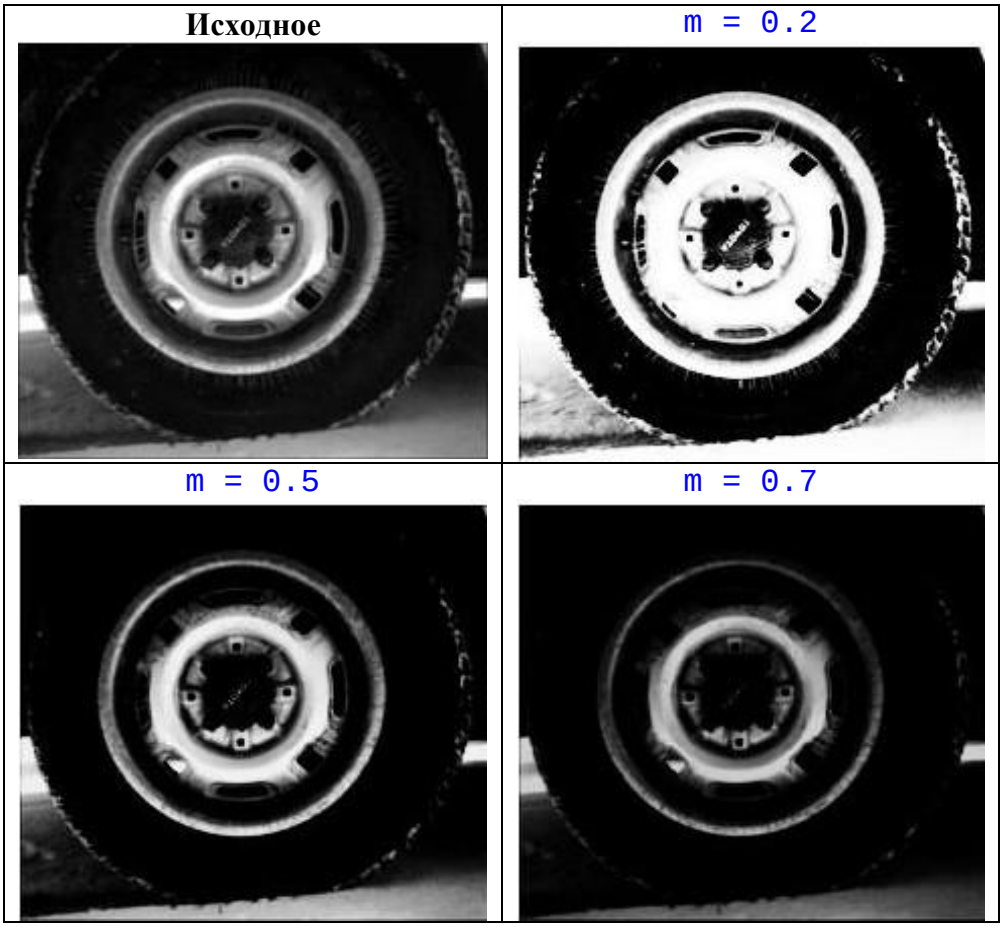
Код системы MATLAB, который позволяет создать эти изображения следующий:

```
I  = imread('tire.tif');
I2 = im2double(I);
m  = mean2(I2);
contrast1 = 1./(1+(m./(I2+eps)).^4);
contrast2 = 1./(1+(m./(I2+eps)).^5);
contrast3 = 1./(1+(m./(I2+eps)).^10);
imshow(I2);
figure, imshow(contrast1);
figure, imshow(contrast2);
figure, imshow(contrast3);
```

Этот второй график показывает, как изменение `m` (используя `E = 4`) влияет на контрастную кривую:



Ниже показано исходное изображение, а также результаты применения трех преобразований, приведенных выше. Значения `m`, используемые ниже, равны `0.2`, `0.5` и `0.7`. Обратите внимание, что значение `0.7` приводит к более темному изображению с меньшим количеством деталей на колесе.



Код системы MATLAB, который позволяет создать эти изображения следующий:

```
I  = imread('tire.tif');
I2 = im2double(I);
contrast1 = 1./(1+(0.2./(I2+eps)).^4)
contrast2 = 1./(1+(0.5./(I2+eps)).^4);
contrast3 = 1./(1+(0.7./(I2+eps)).^4);
imshow(I2);
figure, imshow(contrast1);
figure, imshow(contrast2);
figure, imshow(contrast3);
```

3.5. Функции `intrans` и `changeclass`

Файл `intrans.m` содержит функцию, которая реализует все преобразования яркости изображений, упомянутые выше, за исключением преобразования растяжения контрастности. Вы должны разобраться в коде и выяснить, как включить эту возможность.

Функция `intrans` опирается на вторую функцию, под названием `changeclass`.

Комментарии, начинающиеся со второй строки функции `intrans`, описывают как ее использовать. Обратите внимание, на описание для функции преобразования растяжения контрастности (contrast stretch transform) – она должна принимать различные параметры и какие значения по умолчанию будут использоваться для пропущенных аргументов. В следующей таблице приведены некоторые примеры использования функции `intrans` для соответствующих четырех функций преобразования интенсивности. Предположим, что:

```
I = imread ('tire.tif');
```

Преобразование	Функция преобразования интенсивности	Соответствующий вызов функции `intrans`
Фотографический негатив	`neg = imcomplement(I);`	`neg = intrans(I,'neg');`
Логарифмическое	`I2  = im2double(I);` `log = 5*log(1+I2);`	`log = intrans(I,'log',5);`
Гамма	`gamma = imadjust(I,[],[],0.4);`	`gamma = intrans(I,'gamma',0.4);`
Растяжение контрастности	`I2 = im2double(I);` `contrast=1./(1+(0.2./(I2+eps)).^5);`	`contrast=intrans(I,'stretch',0.2,5);`

2. Пространственная фильтрация

Как уже не раз отмечалось, методы обработки изображений с точки зрения реализации делятся на два основных класса – локальные и глобальные. Каждый из них имеет свои преимущества и недостатки. Преимущество методов при глобальной реализации заключается в простоте их исполнения и быстродействии. Локальные методы владеют более широкими функциональными возможностями, в частности, они могут учитывать характеристики локальных областей, т. е. быть адаптивными.

Image Processing Toolbox обеспечивают универсальную среду для анализа, обработки, визуализации изображений и разработки алгоритмов. Можно восстанавливать некачественные изображения или изображения с помехами, улучшать изображения для повышения четкости, извлекать детали, анализировать формы и текстуры, совмещать два изображения и управлять профилями устройств воспроизведения цвета. Image Processing Toolbox предоставляет широкий спектр средств для цифровой обработки и анализа изображений. Будучи тесно связанным со средой разработки приложений MATLAB, пакет Image Processing Toolbox освобождает человека от выполнения длительных операций кодирования и отладки алгоритмов, позволяя сосредоточить усилия на решении основной научной или практической задачи. MATLAB и пакет Image Processing максимально приспособлен для развития, внедрения новых идей и методов пользователя. Для этого имеется набор сопрягаемых пакетов, направленных на решение всевозможных специфических задач и задач в нетрадиционной постановке. Фильтрация представляет собой технологию модификации или улучшения изображения. Например, существует большое количество фильтров для усиления некоторых особенностей изображения или их удаления. Существует ряд методов, в которых значения пикселей обработанного изображения вычисляются на основании значений окрестных пикселей. Разница между этими методами состоит в том, каким образом учитываются значения соседних пикселей. Отметим, что на основании значений соседних пикселей можно говорить об особенностях локальных окрестностей изображения.

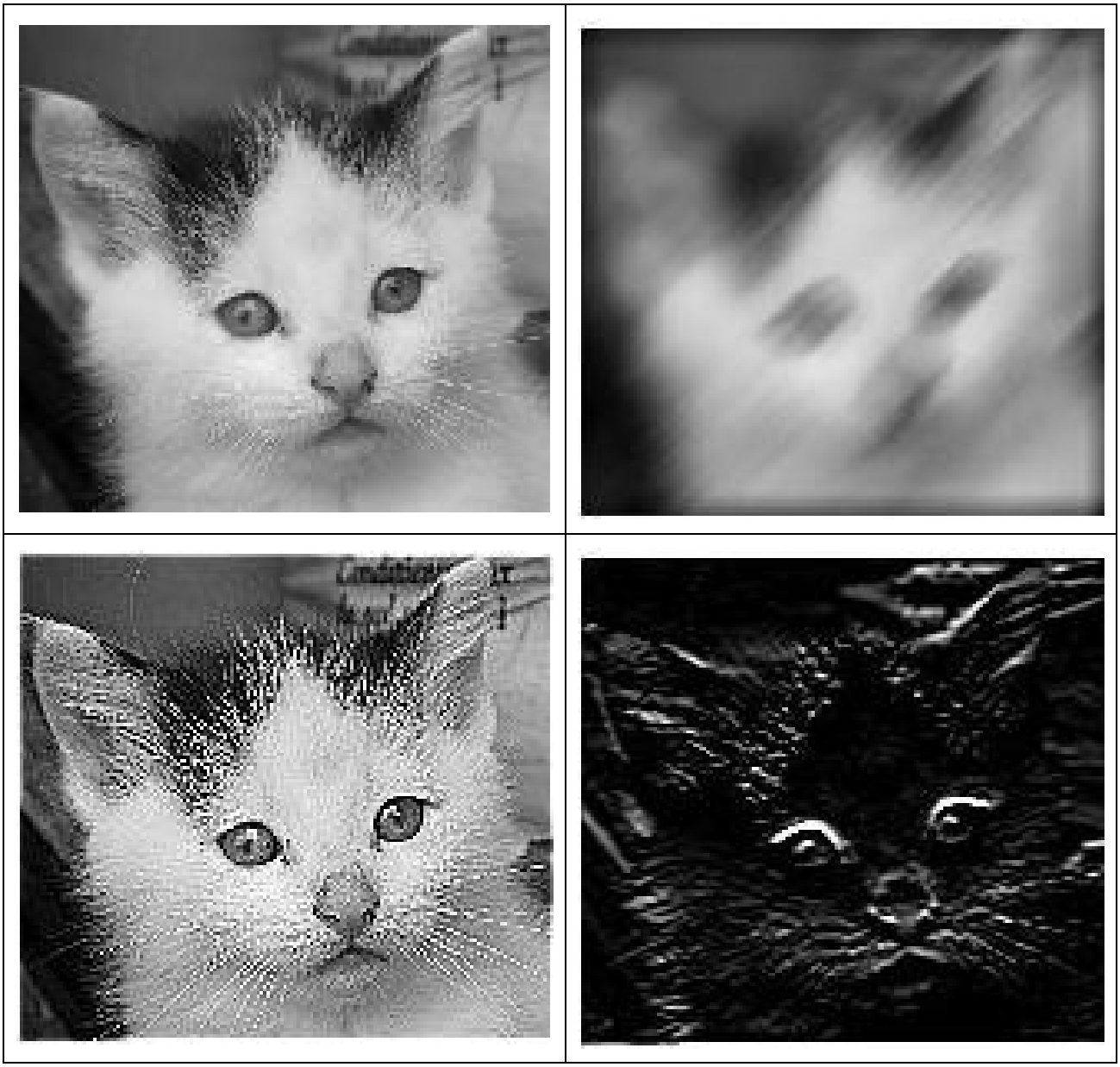
Линейная фильтрация представляет собой такой вид обработки, при которой значения пикселей обработанного изображения формируются в результате линейных операций над значениями пикселей окрестности исходного изображения. Фильтрация с использованием **convolution** (свертки) и **correlation** (корреляции) и функции imfilter довольно часто применяется при обработке изображений.

Существуют два основных типа фильтрации, применяемые к изображениям:

- ✓ пространственная фильтрация;
- ✓ частотная фильтрация.

В дальнейших лабораторных работах мы будем говорить о частотной фильтрации, которые используют преобразования Фурье. Для пространственной фильтрации, мы выполняем операции фильтрации непосредственно над пикселями изображения.

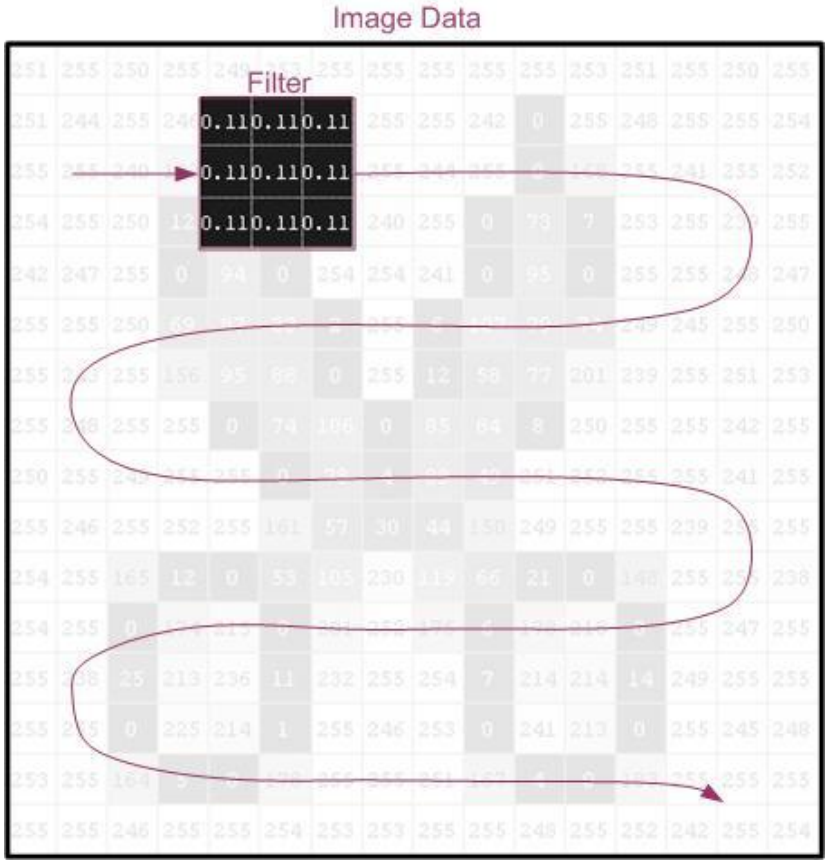
Следующие четыре изображения предназначены для демонстрации того, что может выполнять пространственная фильтрация. Исходное изображение отображено в верхнем левом углу.



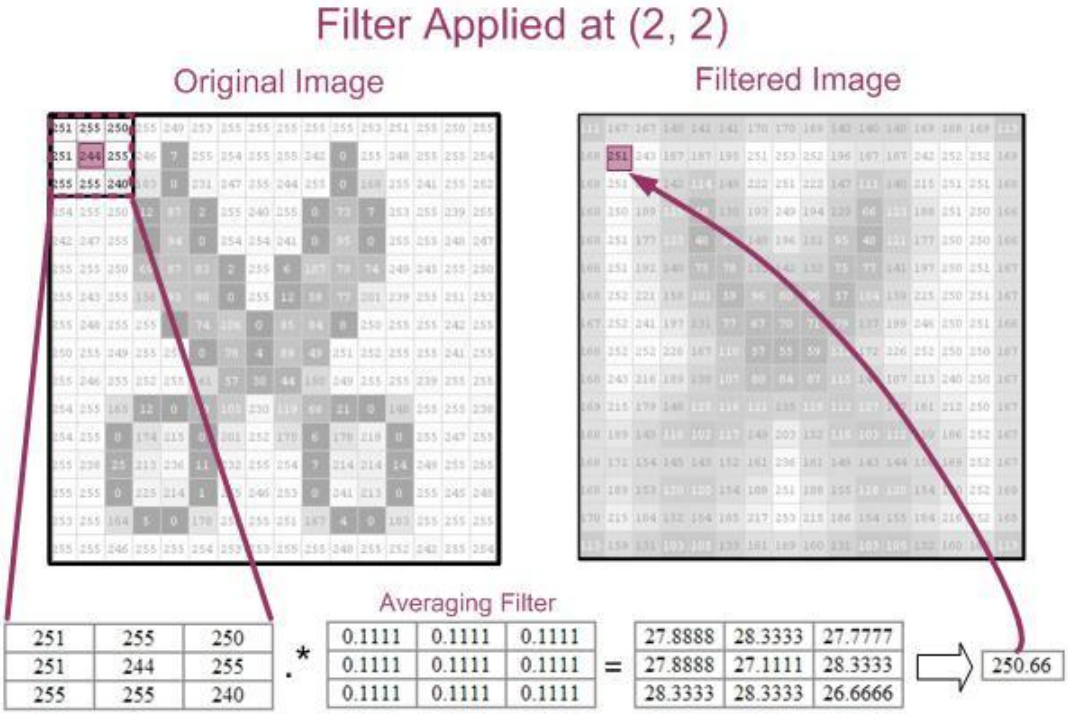
Иногда линейной фильтрации недостаточно для решения конкретной задачи. В этом случае можно использовать высокого порядка математические или полномасштабные MATLAB функции, для производства специализированных результатов. Такая нелинейная фильтрация полезна для сглаживания только гладких областей, улучшения только сильных краев или удаления пятен из изображения.

2.1. Основная идея

Пространственная фильтрация иногда известна также под названием окрестная обработка. Окрестная обработка подходящее имя для нее, потому что определяется центральная точка и выполняется операция (или применяется фильтр) только к тем пикселям которые находятся в некоторой окрестности от этой центральной точки. Результатом операции является единственное значение, которое устанавливается вместо значения, находящегося на месте центральной точки в измененном изображении. Каждая точка на изображении обрабатывается вместе со своими соседними. Общая идея, как показано ниже, так называемого «скользящего фильтра», который перемещается по всему изображению, чтобы вычислить значение в центральной точке.



Следующая диаграмма предназначена для иллюстрации в более подробном виде применение фильтра к изображению. Фильтр (усредняющий фильтр) применяется к пикселю с координатами (2, 2).



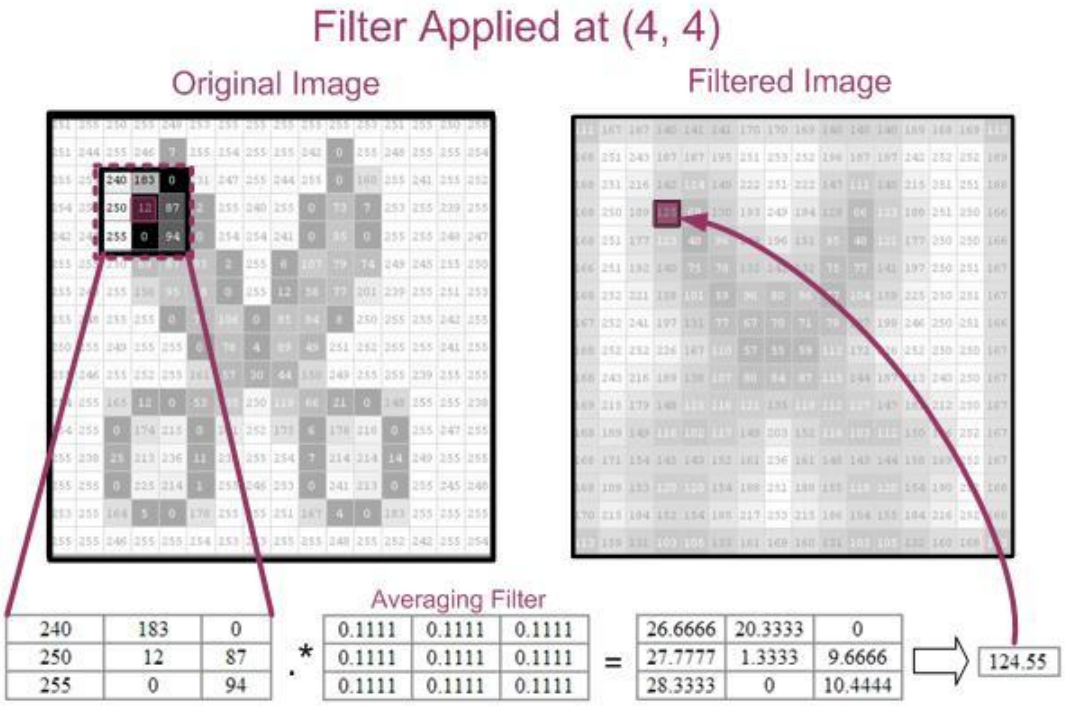
Обратите внимание на то, как вычисленное значение размещается в позиции (2, 2) отфильтрованного изображения.

Ниже приводится описание как результирующее значение 251 (округленное от 250,66) было рассчитано математически:

```
= 251*0.1111 + 255*0.1111 + 250*0.1111 + 251*0.1111 + 244*0.1111 +
```

```
  255*0.1111 + 255*0.1111 + 255*0.1111 + 240*0.1111
= 27.88888 + 28.33333 + 27.77777 + 27.88888 + 27.11111 + 28.33333 +
  28.33333 + 28.33333 + 26.66666
= 250.66
```

Следующий пример показывает применение усредняющего фильтра к пикселю с координатами (4, 4).



Еще раз приводится описание как результирующее значение 125 (округленное от 124,55) было рассчитано математически:

```
= 240*0.1111 + 183*0.1111 + 0*0.1111 + 250*0.1111 + 12*0.1111 + 87*0.1111 +
  255*0.1111 + 0*0.1111 + 94*0.1111
= 26.6666 + 20.3333 + 0 + 27.7777 + 1.3333 + 9.6666 + 28.3333 + 0 + 10.4444
= 124.55
```

Следующая MATLAB функция демонстрирует, как пространственная фильтрация может быть применена к изображению:

```
function img = myfilter(f, w)
%MYFILTER Performs spatial correlation
%   I=MYFILTER(f, w) produces an image that has undergone correlation.
%   f is the original image
%   w is the filter (assumed to be 3x3)
%   The original image is padded with 0's
%Author: Nova Scheidt

% check that w is 3x3
[m,n]=size(w);
if m~=3 | n~=3
    error('Filter must be 3x3')
end

%get size of f
[x,y]=size(f);

%create padded f (called g)
%first, fill with zeros
g=zeros(x+2,y+2);
%then, store f within g
```

```
for i=1:x
    for j=1:y
        g(i+1,j+1)=f(i,j);
    end
end

%cycle through the array and apply the filter
for i=1:x
  for j=1:y
    img(i,j)=g(i,j)*w(1,1)+g(i+1,j)*w(2,1)+g(i+2,j)*w(3,1)...%first column
    + g(i,j+1)*w(1,2)+g(i+1,j+1)*w(2,2)+g(i+2,j+1)*w(3,2)... %second column
    + g(i,j+2)*w(1,3)+g(i+1,j+2)*w(2,3)+g(i+2,j+2)*w(3,3);
  end
end

%Convert to uint--otherwise there are double values and the expected
%range is [0, 1] when the image is displayed
img = uint8(img);
```

Для применения фильтра к примеру выше, были выполнены следующие вызовы (изображение `'stock_cut.jpg'`):

```
w = [1/9 1/9 1/9
     1/9 1/9 1/9
     1/9 1/9 1/9]
stock_cut = imread('stock_cut.jpg');
results = myfilter(stock_cut,w);
imtool(results)
```

2.2. Фильтрация с использованием `imfilter`

Вместо того чтобы использовать M-файлы, приведенные выше, можно использовать функцию, которая поставляется в составе инструментария для обработки изображений (Image Processing Toolkit). Ее можно вызывать таким же образом, как и сценарий MyFilter приведенный выше. В пакете IPT *линейная* пространственная фильтрация реализована функцией `imfilter`, которая имеет следующий синтаксис:

```
g = imfilter(f, w, filtering_mode, boundary_mode, size_options);
```

где `f` – это входное изображение, `w` – фильтрующая маска, g – результат фильтрации, а остальные параметры сведены в таблицу, приведенную ниже. Параметр `filtering_mode` определяет, что совершает фильтр, корреляцию (`'corr'`) или свертку (`'conv'`). Опция `boundary_mode` отвечает за расширение границ, причем размеры расширения определяются размерами фильтра. Опция `size_options` – это либо `'full'`, либо `'same'`.

Чаще всего функция `imfilter` применяется в виде команды:

```
g = imfilter(f, w, 'replicate');
```

Эта форма команды используется при реализации в IPT стандартных линейных пространственных фильтров. Эти фильтры уже повернуты на 180°, поэтому можно делать процедуру корреляции, которая задана в функции по умолчанию. А как уже обсуждалось выше совершение корреляции с перевернутым фильтром эквивалентно свертке с исходным фильтром. Если фильтр симметричен относительно своего центра, то обе опции дают одинаковые результаты.

Функция `imfilter` является более мощной по сравнению со сценарием `myfilter`.

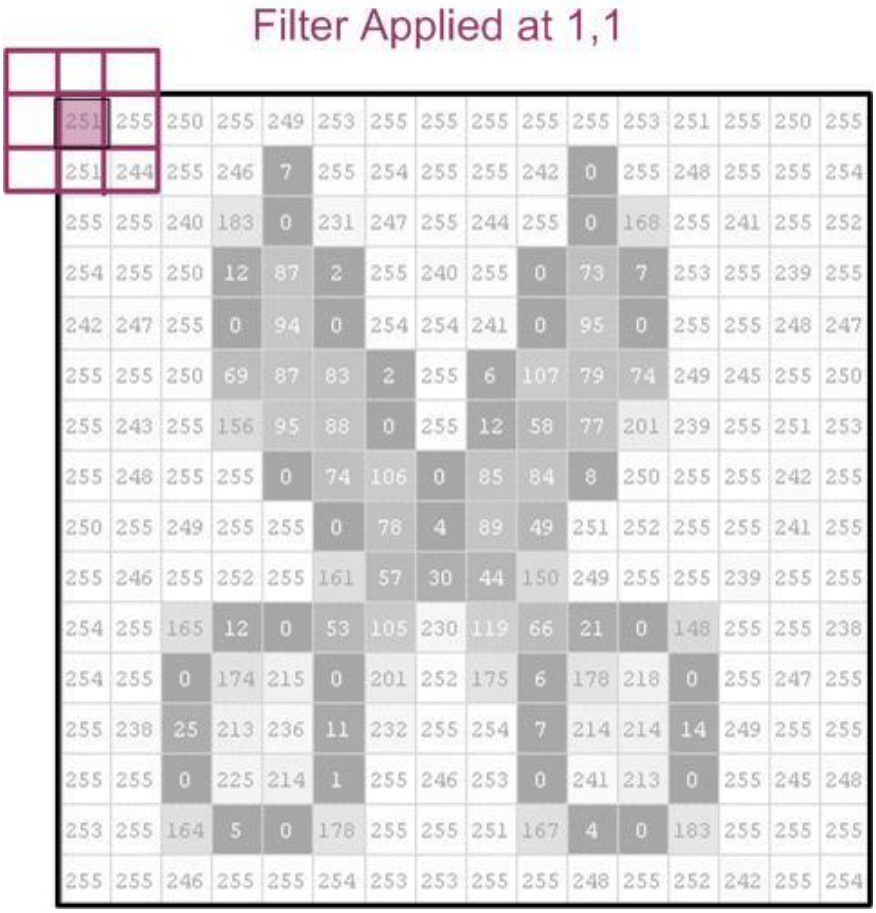
Опции	Описание
Режим фильтрации (Filtering mode)	
`'corr'`	Фильтрация делается методом корреляции. Режим по умолчанию.

`'conv'`	Фильтрация делается методом свертки.
Граничные опции (Boundary mode)	
`P`	Границы изображения расширяются значением `P` (без апострофов). По умолчанию `P = 0`.
`'replicate'`	Размер изображения увеличивается повторением величин на его боковых границах.
`'symmetric'`	Размер изображения увеличивается путем зеркального отражения через границы.
`'circular'`	Размер изображения увеличивается периодическим повторением двумерной функции.
Опции размера (Size options)	
`'full'`	Выход имеет те же размеры, что и расширенное входное изображение.
`'same'`	Выход имеет те же размеры, что и вход. Это достигается с помощью ограничения перемещения центра фильтрующей маски точками, принадлежащими исходному изображению. Опция по умолчанию.

Следующий подпункт обсуждает опции `imfilter`.

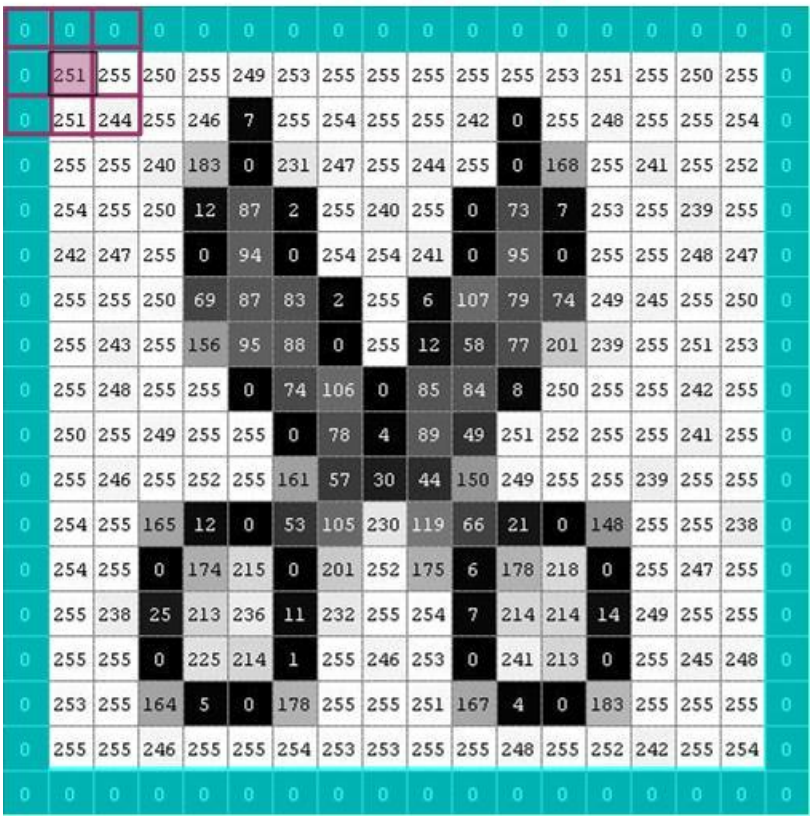
2.2.1. `imfilter` – граничные опции

В приведенном выше примере умышленно был применен фильтр к позиции (2, 2). Это сделано для того, что неотъемлемой проблемой является обработка углов и краев. Проблема в том, что некоторые из «соседей» отсутствуют. Рассмотрим позицию (1, 1):



В этом случае, нет никаких верхних соседей или соседей слева. Существуют два решения данной проблемы: заполнение нулями и репликация (приведены ниже). Пиксели, выделенные голубовато-зеленым оттенком, были добавлены к исходному изображению:

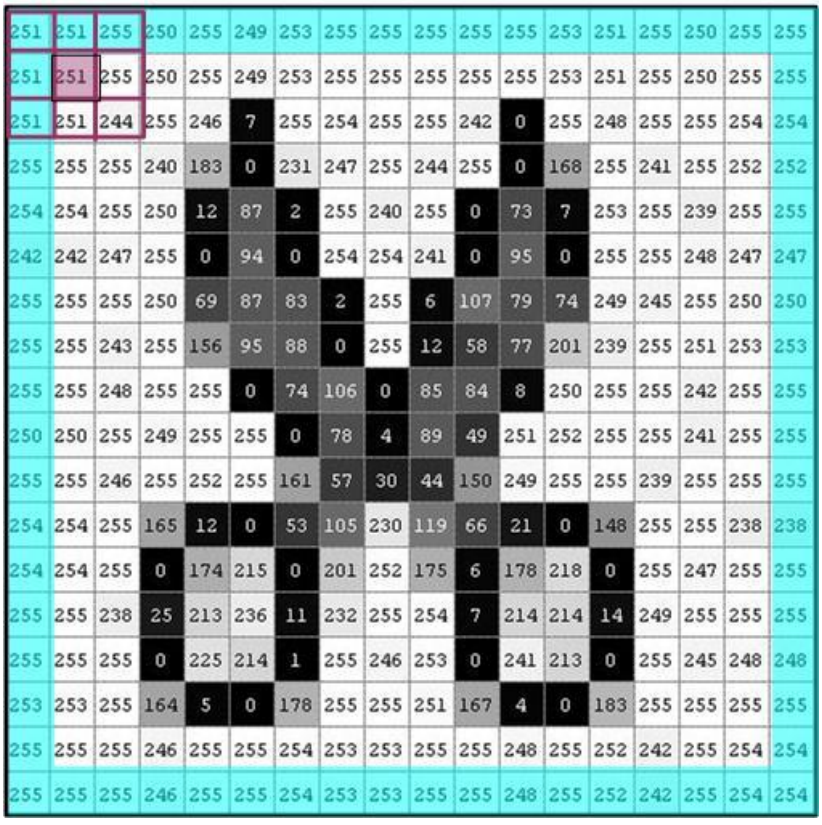
Zero Padding



Заполнение нулями (Zero padding) является по умолчанию. Вы также можете указать значение, отличное от нуля, для использования в качестве значения заполнения.

Другим решением является репликация пикселей вдоль краев:

Replicating



Как памятка, если ваш фильтр был больше, чем 3×3, то «граница заполнения» должна быть расширена. Для фильтра размером 3×3, `'replicate'` и `'symmetric'` дают одинаковые результаты.

На следующих рисунках показаны результаты четырех различных вариантов граничных опций. Фильтр, используемый ниже, является 5×5 усредняющим фильтром, который был создан следующим синтаксисом:

```
h=fspecial('average', 5)
```

Original



Zero-Padded



```
results1 = imfilter(cat,h);
figure, imshow(results1)
title('Zero-Padded')
```

Replicate



```
results2 = imfilter(cat,h,'replicate');
figure, imshow(results2)
title('Replicate')
```

Symmetric



```
results3 = imfilter(cat,h,'symmetric');
figure, imshow(results3)
title('Symmetric')
```

Circular

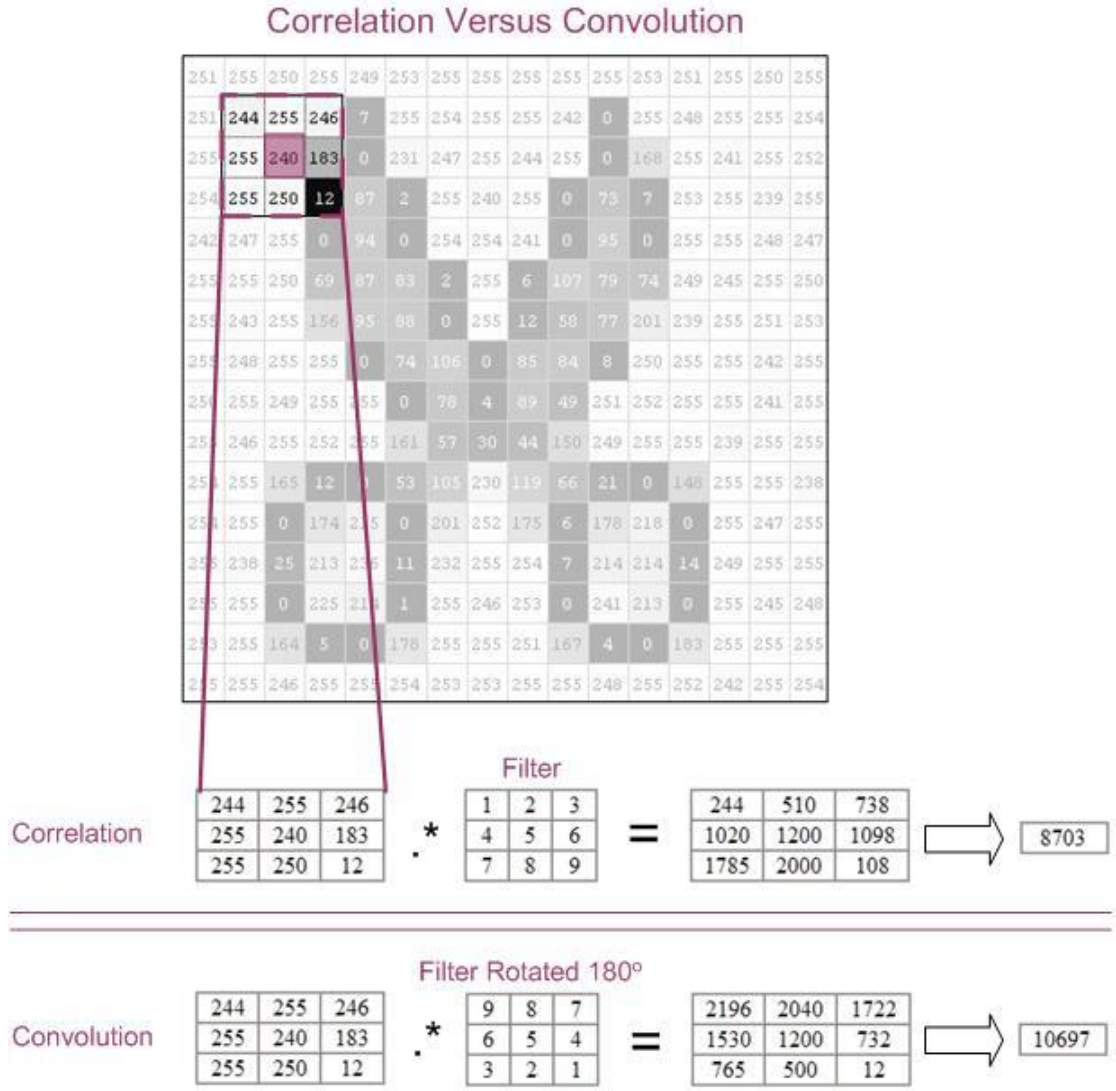


```
results4 = imfilter(cat,h,'circular');
figure, imshow(results4)
title('Circular')
```

Недостаток заполнения нулями заключается в том, что он оставляет темные артефакты вокруг краев обработанного изображения (с белым фоном). Вы это можете увидеть как темную кайму по нижней и правой границе изображения (Zero-Padded), приведенного выше.

2.2.2. `imfilter` – режим фильтрации (Корреляция против Свертки)

С помощью функции `imfilter`, можно выбирать один из двух режимов фильтрации: *корреляцию* или *свертку*. Различие между этими двумя режимами заключается в том, что свертка поворачивает фильтр на 180° перед его применением над изображением. Следующая диаграмма демонстрирует две операции над пикселем изображения с координатами (2, 2):



Этот пример только в демонстрационных целях. Вы заметите, что полученные значения не находятся в интервале [0, 255]. Для получения лучших результатов, вы можете нормализовать фильтр (в данном случае, разделить на 45).

Следующий код демонстрирует MATLAB корреляцию и свертку:

```
h=[1 2 3
   4 5 6
   7 8 9];
h=h/45;
result_corr=imfilter(cat,h);  % correlation is the default,
                              % you can also send 'corr' as an argument
result_conv=imfilter(cat,h,'conv');
```

2.2.3. imfilter – опции размера

Существуют две опции размера `'full'` и `'same'`. Опция `'full'` – результирующий массив содержит полный результат фильтрации. Его размеры больше, чем у исходного массива. Опция `'same'` – размеры результирующего массива совпадают с размерами исходного. Это свойство применяется по умолчанию, когда не указаны опции результирующих размеров.

Для создания `'full'` и `'same'` изображений, можно использовать следующий MATLAB синтаксис:

```
h =[0.1111 0.1111 0.1111
    0.1111 0.1111 0.1111
    0.1111 0.1111 0.1111];
stock_cut_same=imfilter(stock_cut,h);  % 'same' is the default,
                                       % but you can also
                                       % include it as an argument
stock_cut_full=imfilter(stock_cut,h,'full');
```

Если вы используете `imtool` для отображения обоих этих изображений, вы можете заметить, что размер `'same'` 16×16, тогда как `'full'` будет 18×18.

2.3. Предопределенные фильтры

Можно определить фильтры для пространственной фильтрации вручную или можно вызвать функцию, которая будет создавать определенные общие матрицы фильтров. Функция, называемая `fspecial`, принимает аргумент, который определяет требуемый тип фильтра. Полное описание функции `fspecial` доступно в помощи MATLAB:

```
doc fspecial
```

Следующая таблица показывает применение трех фильтров, созданных с помощью функции `fspecial`, над изображением кошки:

Код системы MATLAB	Результирующее изображение

`%исходное изображение`
`cat=imread('cat.jpg');`
`figure, imshow(cat)`
	

`%размывание`
`h=fspecial('motion', 20, 45);`
`cat_motion=imfilter(cat,h);`
`figure, imshow(cat_motion)`
	

```
%увеличение резкости
h=fspecial('unsharp');
cat_sharp=imfilter(cat,h);
figure, imshow(cat_sharp)
```



```
%обнаружение горизонтальных краев
h=fspecial('sobel');
cat_sobel=imfilter(cat,h);
figure, imshow(cat_sobel)
```



3. Упражнения

Часть 1

Закончите `intrans` функцию, реализовав случай с преобразованием растяжения контрастности. Напишите сценарий, который использует ее для воспроизведения 6 изображений, к которым применено преобразование растяжения контрастности, приведенный в пункте 1.4 текущей лабораторной работы

Схема оценивания

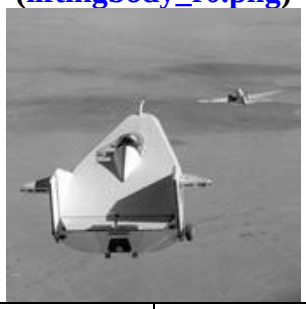
4 очка всего:

- ✓ /3 за добавление функции растяжения контрастности в `intrans`;
- ✓ /1 демонстрация того, как воспроизводятся 6 приведенных изображений.

Часть 2

Определите, какое из приведенных преобразований яркости было использовано над изображением `liftingbody_r0.png` для создания каждого из четырех приведенных ниже результатов. Напишите сценарий для воспроизведения результатов, используя преобразования яркости или их эквивалентов, получаемых с использованием `intrans` функции.

Исходное
(liftingbody_r0.png)



Результат 1 (liftingbody_r1.png)	**Результат 2 (liftingbody_r2.png)**	**Результат 3 (liftingbody_r3.png)**	**Результат 4 (liftingbody_r4.png)**
			

Схема оценивания

12 очков всего, 3 за каждое изображение:

- ✓ /1 определение корректного преобразования яркости;
- ✓ /1 визуальная схожесть результатов;
- ✓ /1 точное совпадение результатов.

Подсказка: каждое приведенное выше преобразование яркости изображения, используется только один раз.

Часть 3: Пространственная фильтрация

1. Загрузите следующее изображение `"two_cats.jpg"` из текущей директории MATLAB.



2. Используйте пространственную фильтрацию для нахождения и отображения горизонтальных краев изображения.
3. Используйте пространственную фильтрацию для нахождения и отображения вертикальных краев изображения. **Подсказка:** прочтите MATLAB документации по функции `fspecial`.
4. Добавьте изображение с горизонтальными краями к изображению с вертикальными краями для получения следующего результата:



Схема оценивания:

6 очков всего:

✓ /2 горизонтальные края;
✓ /2 вертикальные края;
✓ /2 комбинирование результатов.

Конечный результат

✓ Часть 1
  – ваша завершенная `intrans` функция;
  – ваш тестирующий сценарий.
✓ Часть 2
  – сценарий, который создает и отображает ваши версии четырех результирующих изображений.
✓ Часть 3
  – сценарий, который создает и отображает ваши версии четырех контурных изображений, приведенных выше.